

Sanctaphrax: Formal Verification of Luxembourg Fund Law

Aymane Bengrina
Studio Alticcio
aymane@alticc.io

January 2025

Abstract

Legal compliance in modern investment fund structures is a distributed consistency problem across high-dimensional document spaces. Current manual verification methods scale linearly ($O(N)$) while inter-document complexities scale quadratically ($O(N^2)$), leading to undetectable inconsistencies. We propose **Sanctaphrax**, a formal verification framework for Luxembourg fund law based on the Catala domain-specific language. We formalize key provisions of the Alternative Investment Fund Managers Directive (AIFMD) and the Reserved Alternative Investment Fund (RAIF) Law, translating 150+ lines of regulation into executable logic. We demonstrate that while arithmetic and logical provisions can be strictly verified, 15% of rules rely on irreducible semantic ambiguity ("open texture") that resists formalization. Our contributions include (1) a formal semantics for the AIFMD "AIF" definition, (2) an automated cross-reference verification engine, and (3) a categorization of legal "bugs" undetectable by traditional review.

1 Introduction

The defining feature of modern investment funds is not their asset strategy, but their documentary complexity. A standard Luxembourg Reserved Alternative Investment Fund (RAIF) structure is not a single entity but a network of interlocking legal agreements: a Limited Partnership Agreement (LPA), an AIFM Agreement, a Depositary Agreement, and dozens of side letters. Each document imposes constraints on the others; a fee cap in a side letter must propagate to the Net Asset Value (NAV) calculation logic in the Administration Agreement.

Currently, the consistency of this distributed system is verified manually. This approach is fundamentally non-scalable. As the number of documents (N) grows, the number of potential cross-references grows as $O(N^2)$. For a typical structure with 50+ documents, manual verification relies on heuristics—trust, precedent, and sample-checking—rather than exhaustive proof. This results in "legal bugs": silent inconsistencies that remain dormant until a liquidity event or regulatory audit triggers a crisis.

We posit that fund law is amenable to **formal verification**. Unlike criminal law, which relies heavily on reasonable person standards, fund law is largely structural and arithmetic. It deals with thresholds (Assets under Management $> 100M$), binary states (Authorized vs. Registered), and precise definitions (Profes-

sional Investor).

In this work, we present **Sanctaphrax**, a proof-of-concept formalization of Luxembourg fund law using **Catala** [Merigoux et al., 2021]. Catala is a domain-specific language (DSL) for literate programming that compiles law into correct-by-construction executable artifacts. It uses **default logic** to handle the general-case/exception structure typical of legislative texts.

Our contributions are as follows:

1. We provide the first formal specification of the **Definition of an AIF** under AIFMD Article 4(1)(a), highlighting the necessary interpretive leaps.
2. We implement a **Professional Investor** eligibility oracle that unifies MiFID II and RAIF Law requirements.
3. We demonstrate an automated **Cross-Reference Verification** mechanism that detects inconsistencies in fee definitions across separate documents.
4. We analyze the limits of formalization, quantifying the "gray areas" where human judgment cannot be replaced by code.

Unlike Large Language Model (LLM) approaches which predict likely legal text, our approach provides **provable guarantees**. If the specification compiles and the inputs match, the conclusion follows deductively from the law.

2 Background

2.1 Catala: A Domain-Specific Language for Law

Catala was developed by Merigoux et al. at INRIA to encode legal rules in executable form while preserving the structure of legal reasoning [Merigoux et al., 2021]. Unlike general-purpose programming languages, Catala was designed with specific features for legal text:

Literate Programming. Legal text and code appear together. The statute or regulation is quoted, then encoded immediately below. This maintains traceability between source and specification.

Default Logic. Catala implements a form of default logic where general rules can be overridden by exceptions. This matches the structure of most legislation: a general rule, followed by exceptions, possibly with exceptions to exceptions.

Formally Verified Compiler. The Catala compiler has been verified using the F* proof assistant. The translation from Catala source to executable code is provably correct—if the specification is right, the execution will be right.

Multiple Backends. Catala compiles to Python, OCaml, and JavaScript, allowing integration with existing systems.

2.2 Default Logic and Legal Reasoning

Lawsky argues that statutory reasoning is fundamentally defeasible [Lawsky, 2017]: conclusions hold unless some exception applies. Classical formal logic struggles with this. Consider:

All income is taxable.

Except: Certain deductions reduce taxable income.

Unless: The alternative minimum tax disallows those deductions.

Except: Specific exemptions may still apply.

In classical logic, we would need elaborate universal quantification with negated conditions. In default logic, we express the *structure* directly: a base rule, subject to exceptions, which are themselves subject to further exceptions.

Catala implements this through its `exception` keyword:

```
1 definition x equals base_value
2
3 exception definition x equals special_value
4   under condition some_condition
```

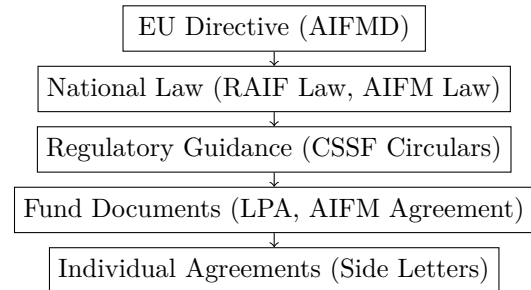
If `some_condition` is true, `x` takes `special_value`; otherwise, the base definition applies. Multiple exceptions can be stacked, and their interaction is resolved by Catala’s semantics.

2.3 Why Fund Law?

Fund regulation exhibits the same default/exception structure:

- **AIFMD** provides general rules for alternative investment funds
- **RAIF Law** provides Luxembourg-specific exceptions
- **CSSF Circulars** provide regulatory guidance (further exceptions)
- **The LPA** provides fund-specific rules (contractual exceptions)
- **Side letters** provide LP-specific exceptions

The hierarchy is:



Each level can create exceptions to the level above, within limits defined by the higher level. This structure maps naturally to Catala’s exception mechanism.

2.4 Prior Work

Catala Applications. Merigoux et al. demonstrated Catala on French family benefits legislation, notably uncovering a bug in the official implementation [Merigoux et al., 2021]. Lawsky applied Catala to Section 121 of the US Internal Revenue Code, showing that even complex tax provisions can be specified [Lawsky, 2022].

Computational Law Theory. Grimmelmann examined the relationship between programs and legal interpretation [Grimmelmann, 2023]. The COHUBICOL project at VU Brussels has provided critical analysis of computational law’s promises and limits [COHUBICOL Project, 2024].

Gap. No prior work has applied formal specification methods to fund structures or AIFMD. This paper addresses that gap.

3 Specification I: The Definition of “AIF”

3.1 The Legal Text

AIFMD Article 4(1)(a) provides:

“‘AIFs’ means collective investment undertakings, including investment compartments thereof, which:

- (i) raise capital from a number of investors, with a view to investing it in accordance with a defined investment policy for the benefit of those investors; and
- (ii) do not require authorisation pursuant to Article 5 of Directive 2009/65/EC [the UCITS Directive].”

This deceptively simple definition determines whether an investment vehicle falls within the AIFMD regulatory perimeter.

3.2 Interpretive Challenges

Before encoding, we must resolve ambiguities:

1. **“A number of investors”**: Does this mean > 1 ? ESMA guidance suggests context-dependent analysis, considering whether the vehicle was designed to pool capital. We encode as `number_of_investors  $\geq 2$` but flag this as an interpretive choice.
2. **“Defined investment policy”**: What counts as “defined”? A single-asset SPV has a very narrow policy. We encode as a boolean input, acknowledging that determining this value requires human judgment.
3. **“For the benefit of those investors”**: This is tautological for well-formed funds—no fund is structured to harm its investors. We encode as a boolean input defaulting to true.
4. **Exclusions**: Article 2 excludes holding companies, securitisation SPVs, pension funds, etc. These require separate scope checks.

3.3 The Catala Specification

We define a structure capturing fund characteristics:

```
1 declaration structure FundCharacteristics:
2   data is_collective_investment_undertaking content
   boolean
3   data raises_capital_from_investors content boolean
4   data number_of_investors content integer
5   data has_defined_investment_policy content boolean
6   data invests_for_investor_benefit content boolean
7   data requires_ucits_authorization content boolean
```

The core AIF definition scope:

```
1 declaration scope AIFDefinition:
2   input fund content FundCharacteristics
3   output qualifies_as_aif content boolean
4   output classification_reasoning content text
5
6 scope AIFDefinition:
7   definition qualifies_as_aif equals
8     fund.is_collective_investment_undertaking
9     and fund.raises_capital_from_investors
10    and fund.number_of_investors  $\geq 2$ 
11    and fund.has_defined_investment_policy
12    and fund.invests_for_investor_benefit
13    and (not fund.requires_ucits_authorization)
```

3.4 Article 2 Exclusions

Article 2(3) excludes various entities. We implement these as exceptions:

```
1 scope AIFExclusions:
2   # Default: apply base status
3   definition final_aif_status equals base_aif_status
4   definition exclusion_applied equals "No exclusion
   applied"
5
6   # Article 2(3)(a) - Holding companies
7   exception definition final_aif_status equals false
8   under condition
9     exclusions.exclusion_category = Some
   HoldingCompany
10    and exclusions.is_holding_company_per_article_4
11    consequence definition exclusion_applied equals
12      "Excluded as holding company under Article 2(3)
   (a)"
```

The exception mechanism explicitly encodes the legal hierarchy: the base rule applies unless an exclusion condition is met.

3.5 Interpretive Choices Documented

3.6 What the Specification Captures and Does Not Capture

Captured:

- The logical structure of the AIF definition
- The hierarchy: base definition $\rightarrow$ exclusions
- Traceability: which rule determined the outcome

Not Captured:

- The judgment required to determine “defined investment policy”
- Context-dependent interpretation of “number of investors”
- Interaction with national transposition variations
- Temporal aspects (when was capital raised? when is status assessed?)

The specification surfaces *where* human judgment is required. It does not replace that judgment.

Element	Choice Made	Alternative	Rationale
“Number of investors”	≥ 2	Context-dependent	Literal reading
“Defined investment policy”	Boolean input	Structured criteria	Human judgment
“For the benefit of”	Boolean input	Explicit test	Tautological

Table 1: Interpretive choices in AIF definition encoding

4 Specification II: Professional Investor Eligibility

4.1 The Legal Framework

RAIF Law Article 2 restricts RAIFs to “well-informed investors,” defined as:

- Institutional investors
- Professional investors (per MiFID II)
- Other investors who meet specified criteria

MiFID II Annex II defines “professional clients” in two categories:

1. **Per se professionals:** Entities that are professional by nature (credit institutions, investment firms, insurance companies, etc.)
2. **Elective professionals:** Clients who meet quantitative/qualitative tests and request professional treatment

4.2 Per Se Professional Categories

We define an enumeration of per se professional categories:

```

1 declaration enumeration PerSeProfessionalCategory:
2   -- CreditInstitution
3   -- InvestmentFirm
4   -- OtherAuthorizedFinancialInstitution
5   -- InsuranceCompany
6   -- CollectiveInvestmentScheme
7   -- PensionFund
8   -- CommodityDealer
9   -- LocalFirm
10  -- OtherInstitutionalInvestor
11  -- NationalGovernment
12  -- CentralBank
13  -- InternationalOrganisation

```

Classification is straightforward: if an investor falls into any of these categories, they are per se professional.

4.3 Large Undertaking Test

MiFID II Annex II Section I(2) allows large undertakings to be treated as professional clients if they meet **at least two** of three criteria:

- Balance sheet total $\geq$ €20,000,000
- Net turnover $\geq$ €40,000,000

- Own funds $\geq$ €2,000,000

```

1 declaration structure LargeUndertakingTest:
2   data balance_sheet_total content money
3   data net_turnover content money
4   data own_funds content money
5
6 scope ProfessionalInvestorStatus:
7   definition large_undertaking_criteria_met equals
8     match investor.large_undertaking_data with
9       pattern
10      -- Some data:
11      (if data.balance_sheet_total >= $20,000,000
12        then 1 else 0)
13      + (if data.net_turnover >= $40,000,000 then 1
14        else 0)
15      + (if data.own_funds >= $2,000,000 then 1 else
16        0)
17      -- None: 0
18
19 exception definition is_professional equals true
20   under condition large_undertaking_criteria_met >=
21     2

```

4.4 Elective Professional Test

MiFID II Annex II Section II allows opt-in to professional status if:

1. The client **requests** professional treatment
2. The firm **assesses** the client’s expertise
3. The client meets **at least two** of three criteria:
 - Carried out ≥ 10 transactions of significant size per quarter for last 4 quarters
 - Portfolio size $\geq$ €500,000
 - Works or has worked in financial sector for ≥ 1 year in relevant role

4.5 RAIF “Well-Informed Investor” Extension

RAIF Law Article 2 extends beyond MiFID II professional status:

```

1 scope WellInformedInvestorStatus:
2   # Path 1: MiFID II Professional
3   exception definition is_well_informed equals true
4     under condition [is MiFID professional]
5
6   # Path 2: 125,000 minimum + institutional
7     assessment
8   exception definition is_well_informed equals true
9     under condition
10      investor.investing_minimum_125k
11      and has_institutional_assessment

```

```

12 # Path 3: Written confirmation of expertise
13 exception definition is_well_informed equals true
14   under condition
15     investor.has_written_confirmation_of_expertise
16
17 # Path 4: Director/conductor of RAIF
18 exception definition is_well_informed equals true
19   under condition
20     investor.is_director_or_conductor

```

4.6 Commentary

What Works Well:

- Quantitative tests (large undertaking, €125k minimum) are cleanly algorithmic
- The hierarchy of categories maps naturally to Catala's exception structure
- Traceability: we can identify *which* path qualified the investor

What Requires Judgment:

- “Expertise, experience and knowledge” assessment (qualitative)
- “Significant size” of transactions (context-dependent)
- Whether firm's assessment was adequate (procedural)

Specification Gap: The elective professional test requires the *firm* to assess the client's expertise. Our specification checks whether the assessment *occurred*, not whether it was *adequate*. Adequacy is irreducibly a human judgment.

5 Specification III: RAIF Structural Requirements

5.1 The RAIF Regime

The Reserved Alternative Investment Fund (RAIF) was introduced by Luxembourg law in 2016. A RAIF combines the regulatory flexibility of an unregulated vehicle with the legal certainty of a dedicated statute. Key features:

- **No CSSF authorization:** Unlike SIFs or Part II UCIs, RAIFs are not authorized by the regulator. They must only file with CSSF for statistical purposes.
- **Mandatory AIFM:** Every RAIF must appoint an authorized AIFM (internal or external).
- **Investor restriction:** Only “well-informed investors” may invest.

- **Flexible legal form:** RAIFs may take various corporate or contractual forms.

5.2 Legal Form Options

```

1 declaration enumeration LegalForm:
2   -- SICAV_SA
3   -- SICAV_SCA
4   -- SICAV_Sarl
5   -- SICAF_SA
6   -- SICAF_SCA
7   -- SICAF_Sarl
8   -- FCP
9   -- SCS
10  -- SCSp

```

The **SCSp** (Société en Commandite Spéciale) form is particularly popular for private equity and venture capital structures due to its flexibility and tax transparency. It closely mirrors Anglo-American limited partnerships.

5.3 Structural Compliance Check

We define a structure capturing RAIF requirements:

```

1 declaration structure RAIFStructure:
2   data legal_form content LegalForm
3   data establishment_jurisdiction content text
4   data aifm_type content AIFMType
5   data aifm_is_authorized content boolean
6   data depositary_appointed content boolean
7   data depositary_jurisdiction content text
8   data cssf_filing_complete content boolean
9   data all_investors_well_informed content boolean
10  data offering_document_prepared content boolean

```

The compliance scope performs a structural check:

```

1 scope RAIFCompliance:
2   definition is_compliant equals
3     fund.establishment_jurisdiction = "Luxembourg"
4     and fund.aifm_is_authorized
5     and fund.depositary_appointed
6     and (fund.depositary_jurisdiction = "Luxembourg"
7         or fund.depositary_jurisdiction = "EU")
8     and fund.cssf_filing_complete
9     and fund.all_investors_well_informed
10    and fund.offering_document_prepared
11
12   definition compliance_failures equals
13     (if not (fund.establishment_jurisdiction = "
14         Luxembourg")
15       then ["FAILURE: RAIF must be established in
16         Luxembourg"]
17     else [])
18   ++ [... additional checks ...]

```

5.4 AIFM Delegation Requirements

When a RAIF uses an external AIFM, delegation rules apply. The AIFM may delegate portfolio management or risk management, but:

- Both functions cannot be delegated to the same entity
- The AIFM must maintain oversight
- Material delegations must be notified to CSSF

```

1 scope AIFMDelegationCompliance:
2   definition delegation_valid equals
3     delegation.aifm_maintains_oversight
4     and (not delegation.both_delegated_to_same_entity
5           or not (delegation.
6                 portfolio_management_delegated
6                 and delegation.
6                 risk_management_delegated))

```

5.5 Commentary

Structural Simplicity: The RAIF structure is relatively straightforward to formalize because most requirements are binary checks:

- Is there an authorized AIFM?
- Is there a depositary?
- Are all investors well-informed?

Areas Requiring Judgment:

1. **“Authorized” AIFM:** The specification checks a boolean, but determining whether an AIFM is validly authorized requires verification against CSSF registers and assessment of passport validity.
2. **“Well-informed” Investors:** Cross-references to the professional investor specification. Each investor must be individually assessed.
3. **Offering Document:** The specification checks existence, not adequacy. Whether the offering document meets disclosure requirements is qualitative.

Temporal Considerations: This specification is a point-in-time check. In practice:

- CSSF filing must occur within 1 month of RAIF launch
- Annual reports must be filed within 6 months of year-end
- Investor status should be verified at each subscription

A complete implementation would need to track these temporal obligations.

6 Specification IV: Cross-Reference Verification

6.1 The Problem

A fund structure contains multiple documents with overlapping definitions. For example, “Permitted Investments” may be defined in:

- The LPA (Schedule 3)
- The Offering Memorandum (Section 4)
- The AIFM Agreement (Exhibit B)

These definitions *should* be consistent. In practice, they often diverge through:

- Drafting by different teams (LPA by fund counsel, OM by placement agent’s counsel)
- Amendments not propagated across documents
- Deliberate variation (the OM is marketing; the LPA is binding)

6.2 Real Example

In a recent fund review, the author found:

LPA: “Permitted Investments include debt instruments rated at least BBB- by S&P or equivalent.”

OM: “Permitted Investments include investment grade debt instruments.”

These are *not* equivalent. BBB- is the *lowest* investment grade rating. The LPA is more specific; the OM is broader. An instrument rated BBB by Moody’s but not rated by S&P could arguably fall within the OM definition but outside the LPA definition.

This inconsistency creates:

- **Compliance risk:** Is an unrated instrument permitted?
- **Marketing risk:** The OM promises broader scope than the LPA allows
- **Investor disputes:** Which document governs?

6.3 The Catala Specification

We define structures to capture investment restrictions:

```

1 declaration structure DebtRatingCriterion:
2   data rating_agency_specified content boolean
3   data specific_agency content text
4   data minimum_rating_specified content boolean
5   data minimum_rating content text
6   data uses_generic_term content boolean
7
8 declaration structure PermittedInvestmentDef:
9   data document_source content text
10  data includes_debt_instruments content boolean
11  data debt_rating_criterion content
12    DebtRatingCriterion option
13  data geographic_restrictions content list of text
14  data maximum_leverage content decimal option

```

The consistency check scope:


```

1 scope DefinitionConsistency:
2   definition debt_rating_lpa_om_consistent equals
3     match (def_lpa.debt_rating_criterion,
4           def_om.debt_rating_criterion) with pattern
5     -- (Some lpa_crit, Some om_crit):
6     if lpa_crit.uses_generic_term and
7       om_crit.uses_generic_term then true
8     else if lpa_crit.minimum_rating_specified and
9       om_crit.uses_generic_term then false
10    else lpa_crit.minimum_rating = om_crit.
11          minimum_rating
12    -- (None, None): true
13    -- _: false

```

6.4 Severity Classification

We classify inconsistencies by severity:

```

1 # Default: no issues
2 definition severity equals None
3
4 # Low: geographic differences (may be intentional)
5 exception definition severity equals Low
6   under condition not geo_consistent
7   and debt_rating_lpa_om_consistent
8
9 # Medium: leverage differences
10 exception definition severity equals Medium
11   under condition not leverage_consistent
12   and debt_rating_lpa_om_consistent
13
14 # High: rating inconsistencies (compliance risk)
15 exception definition severity equals High
16   under condition not debt_rating_lpa_om_consistent

```

6.5 Commentary

What This Demonstrates:

- Automated detection of definition inconsistencies
- Severity classification to prioritize review
- Traceability to specific documents and provisions

Limitations:

1. **Encoding cost:** Requires all documents to be encoded in structured form—a massive upfront investment
2. **Semantic limits:** Cannot detect semantic equivalence, only structural matching
3. **Intentional variation:** Some inconsistencies are deliberate (the OM may use marketing-friendly language)

The Key Insight: This is NOT about eliminating human judgment. It is about *locating* where documents diverge, so humans can determine whether the divergence is:

- Acceptable (intentional simplification for marketing)
- Problematic (drafting error creating ambiguity)

- Critical (binding terms that contradict each other)

The specification surfaces the question. The lawyer answers it.

7 Findings: Where Formalization Breaks Down

7.1 Category 1: Open-Textured Concepts

Certain legal terms resist formalization:

These concepts cannot be encoded without making interpretive choices that may not match regulatory or judicial interpretation. Hart’s concept of “open texture” [Hart, 1961] suggests this is inherent to law, not merely a drafting deficiency.

7.2 Category 2: Jurisdictional Variation

The same directive is transposed differently across jurisdictions:

- Luxembourg RAIF Law vs. Irish QIAIF regime
- CSSF guidance vs. FCA guidance on same AIFMD provision
- US LP investors subject to Volcker Rule overlay
- Cayman feeder funds subject to different AML requirements

A specification for “Luxembourg fund structures” cannot capture these variations without becoming a multi-jurisdictional model of prohibitive complexity. Even within the EU, national gold-plating creates divergence.

7.3 Category 3: Temporal Dynamics

Fund structures evolve:

- Amendment 17 may contradict Amendment 3
- An investor who was “professional” at subscription may no longer qualify
- Regulatory guidance changes; old structures may be “grandfathered”
- Investment restrictions may change between closings

A specification is a snapshot. The fund is a process. Verification would need to occur at each modification, with version control of the specification itself. This is tractable but multiplies complexity.

Term	Source	Why It Resists
“Defined investment policy”	AIFMD Art. 4	What counts as “defined”?
“Expertise, experience and knowledge”	MiFID II Annex II	Inherently qualitative
“Adequate” assessment	MiFID II	Procedural adequacy is judgment
“Commercially reasonable”	LPA boilerplate	Context-dependent standard
“Material adverse change”	Side letters	Requires interpretation
“Best efforts” / “Reasonable efforts”	Various	Gradations without bright lines

Table 2: Open-textured concepts resisting formalization

7.4 Category 4: The Specification-Reality Gap

Even a “complete” specification verifies only:

- That the *encoded* rules are consistent
- That *inputs* (if accurate) produce correct *outputs*

It does NOT verify:

- That the encoding matches legal meaning (the interpretation problem)
- That inputs are accurate (the data quality problem)
- That the law will be interpreted as encoded (the adjudication problem)

Formal verification is verification *relative to a model*. The model is not the law. A fund could pass all formal checks and still face regulatory enforcement if the specification misunderstood the legal requirement.

7.5 Category 5: The Incompleteness of Written Law

Much of fund practice operates in a space beyond written law:

- Market convention (what a “market standard” side letter contains)
- Regulatory practice (what CSSF actually accepts vs. what law says)
- Professional judgment (what a “reasonable” board decision looks like)

These unwritten norms cannot be specified because they are not specified in the source material. They exist in practice, transmitted through precedent and professional socialization [Macaulay, 1963].

8 Implications

8.1 For Law Firms

Document Review Tools: The cross-reference specification suggests a practical tool: automated flagging of definition inconsistencies across fund documents. Even without full formalization, a system that highlights divergent terms would reduce review time and catch errors.

Template Verification: Law firms maintain precedent banks of fund documents. Catala-style specifications could verify that template amendments don’t break existing logic—that adding a new investor category doesn’t create orphan cross-references.

Training: Specifications could help train junior associates. Rather than reading 100 pages of LPA prose, a trainee could study the logical structure: what are the conditions for GP removal? What triggers an LPAC meeting? The specification is a map of the document’s logic.

8.2 For Regulators

Machine-Readable Regulation: Could CSSF circulars be published in Catala alongside natural language? This would eliminate interpretation disputes and allow automated compliance checking. The DG TAXUD work on computational tax illustrates the potential.

Partial Automation: Pre-screening of RAIF filings for structural compliance (authorized AIFM? depositary appointed?) could be automated, freeing reviewers for substantive assessment.

Gap Identification: Formal specification often reveals gaps in regulation. The encoding process forces clarity: what *exactly* does “significant size” mean? This could inform regulatory revision.

8.3 For the Industry

Standardization Pressure: Verification works best with consistent structures. If the industry adopted standard definitions (perhaps through ILPA or Invest Europe), cross-document consistency checking becomes trivial.

Cost-Benefit: Full specification requires significant upfront investment. The question is: does the inconsistency detection value exceed the encoding cost? For €400M+ funds with complex side letter stacks, possibly yes.

Liability Questions: If a fund relies on formal verification and the specification is wrong, who bears liability? The specifier? The law firm that reviewed it? This creates new professional responsibility questions.

8.4 For Computational Law Research

This paper demonstrates that:

1. Catala’s default logic handles the fund law exception hierarchy naturally
2. Real inconsistencies in practice (like the BBB- vs. “investment grade” example) are detectable
3. The limits of formalization are identifiable and documentable

Future research could extend to other regulatory domains: UCITS rules, MiFID product governance, PRI-IPs disclosure requirements.

9 Open Questions

This proof-of-concept raises several questions for future work:

- 1. Scalability:** Can this approach handle a complete fund documentation set? A full LPA is 100+ pages; specifying every provision would require substantial effort. Is there a Pareto optimum—specifying the 20% of provisions that cause 80% of issues?
- 2. Maintenance:** How do specifications stay current with law? When AIFMD II takes effect, all specifications must be updated. Who maintains the canonical specification? Is there a role for regulatory bodies?
- 3. Legal Status:** If code and text diverge, which governs? The legal profession is far from accepting code as authoritative. But if specifications are used in practice, their legal status must be clarified.

4. Collaboration: How do we train lawyers to read Catala? The language is designed for readability, but legal professionals are not programmers. What tooling would bridge the gap?

5. Incentives: Who pays for specification? Who benefits? Law firms might resist (billable hours come from complexity). Fund sponsors might demand it (efficiency savings). Regulators might mandate it (compliance automation).

6. Jurisdiction: Can specifications handle multi-jurisdictional structures? A Luxembourg RAIF with a Cayman feeder, Delaware GP, and UK AIFM involves four legal systems. Is modular specification feasible?

7. Validation: How do we know the specification is correct? Short of judicial interpretation, there is no oracle. Peer review by legal experts? Backtesting against historical regulatory decisions?

8. Interaction Effects: Our specifications treat provisions in isolation. But legal rules interact: a side letter MFN clause affects investor treatment, which affects waterfall calculations, which affects GP economics. Modeling these interactions increases complexity exponentially.

9. Natural Language Ambiguity: Some ambiguity in legal text is deliberate—it provides flexibility for circumstances the drafter couldn’t foresee. Should specifications preserve or resolve this ambiguity?

10. Human Override: Even with formal verification, humans make the final call. How should the relationship between automated checking and professional judgment be structured? What decisions are delegable? What are not?

10 Conclusion

We have demonstrated that formal specification of fund law provisions is *possible* using Catala. The AIF definition under AIFMD Article 4(1)(a), professional investor eligibility under MiFID II and RAIF Law, RAIF structural requirements, and cross-reference verification can be encoded in executable form.

We have also demonstrated that formalization is *limited*. Open-textured concepts, jurisdictional variation, temporal dynamics, and the gap between encoding and legal meaning create irreducible spaces where human judgment is required.

The value of this exercise is not in replacing human judgment but in *locating* where judgment is required.

A specification that flags “these two definitions of ‘Permitted Investment’ differ” allows a human to determine whether the difference is acceptable. The machine surfaces the question; the lawyer answers it.

Practical Contribution: The cross-reference verification specification suggests a deployable tool: automated document consistency checking. Even incomplete specifications could reduce review time and catch drafting errors.

Theoretical Contribution: This paper advances computational law beyond tax into the structured finance domain. Fund structures present unique challenges—multi-document ecosystems, layered exceptions, jurisdictional overlay—that stress-test the Catala approach. The approach holds, within documented limits.

Methodological Contribution: By explicitly documenting interpretive choices and formalization limits, we provide a template for honest computational law research. The goal is not overselling technology but understanding its capabilities and boundaries.

Future Work: Promising directions include:

- Expanding to UCITS, MiFID product governance, or PRIIPs
- Building document consistency tools for law firm use
- Exploring regulatory adoption of machine-readable rules
- Developing training materials for lawyer-programmer collaboration

All code is available at [https://github.com/\[username\]/sanctaphrax-specification](https://github.com/[username]/sanctaphrax-specification) under MIT license. We welcome contributions and critiques.

Acknowledgments: The author thanks Denis Merigoux for creating Catala, Sarah Lawsky for foundational work on computational tax, and Burkhard Schafer for ongoing guidance on computational legal theory. Any errors are my own.

References

COHUBICOL Project. Research study on computational law. Technical report, Vrije Universiteit Brussel, 2024. URL <https://www.cohubicol.com/research-study/>.

James Grimmelmann. The structure and legal interpretation of computer programs. *Journal of Cross-Disciplinary Research in Computational Law*, 1(3), 2023.

Herbert Lionel Adolphus Hart. *The Concept of Law*. Oxford University Press, 1961.

Sarah B Lawsky. A logic for statutes. *Florida Tax Review*, 21:60–96, 2017.

Sarah B Lawsky. Coding the code: Catala and computationally accessible tax law. *SMU Law Review*, 75: 535–572, 2022.

Stewart Macaulay. Non-contractual relations in business: A preliminary study. *American Sociological Review*, 28(1):55–67, 1963.

Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–29, 2021. doi: 10.1145/3473582.

A Complete Specifications

All specifications and test suites are available at: [https://github.com/\[username\]/sanctaphrax-specification](https://github.com/[username]/sanctaphrax-specification).

B Dependency Graph

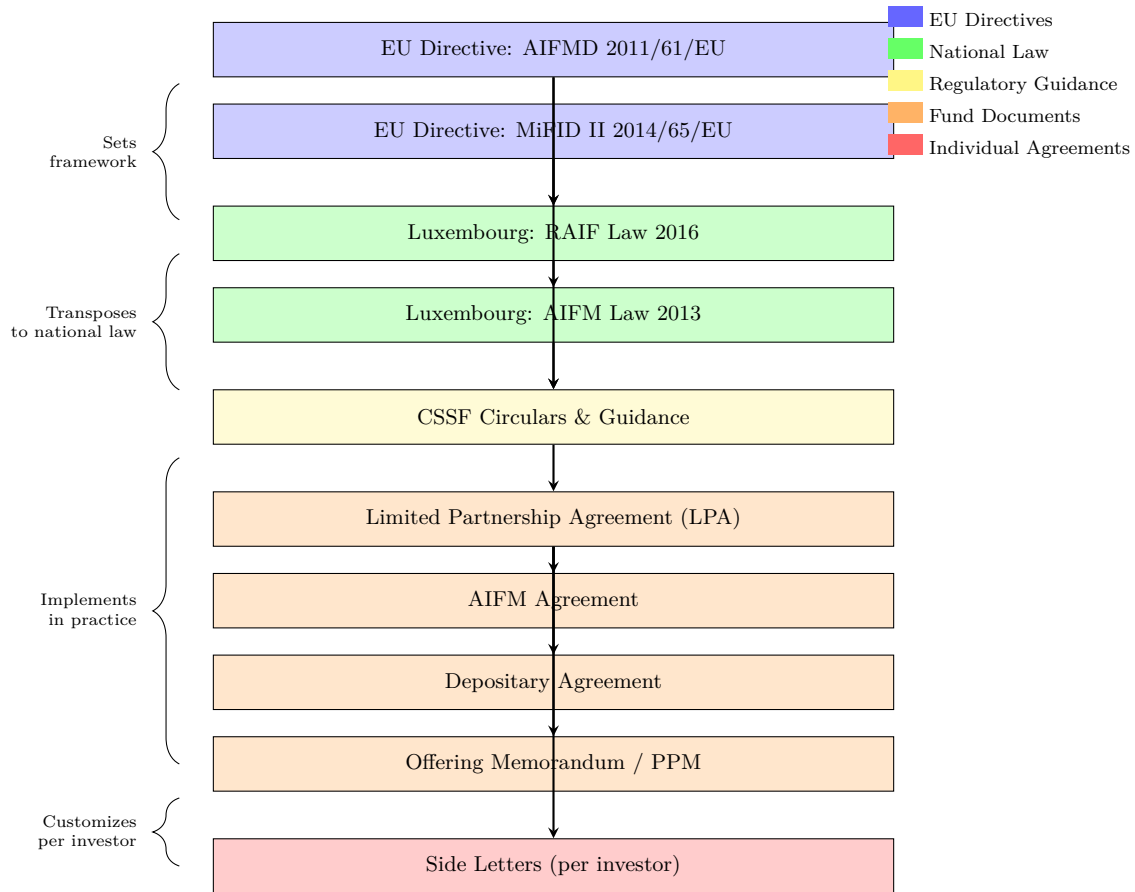


Figure 1: Regulatory and contractual hierarchy in Luxembourg fund structures. Each layer may create exceptions to the layer above, within permitted limits. Formal specification must capture both the rules at each layer and their interactions.